Minimizing Logic Gates in an FP4×FP4 Multiplier: A Structural Decomposition Approach

Chaithu Talasila
Etched Take-Home Assignment
talasila.chaithu1@gmail.com

April 27, 2026

Abstract

We present a **86-gate** Boolean circuit that correctly computes the product of two 4-bit floating-point (FP4, E2M1 format) numbers and returns the result as a 9-bit fixed-point integer (QI9, where the LSB has value $\frac{1}{4}$). The circuit uses only two-input AND, OR, XOR gates and one-input NOT gates, each counted as one gate. Starting from a naïve Quine-McCluskey flat-SOP baseline of approximately 288 gates, we apply a series of mathematically motivated transformations that exploit the product structure of the FP4 format, ultimately reducing gate count by 70%. We document each optimization step, prove impossibility of several sub-expression classes, and provide an estimated lower bound of 75–85 gates, suggesting that further reductions of at most ∼10 gates are possible.

1 Introduction

Modern machine learning accelerators increasingly use sub-8-bit floating-point formats [1, 2]. The FP4 E2M1 format (4-bit: 1 sign, 2 exponent, 1 mantissa, no NaN/Inf) represents 16 values: $0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6$. Etched's take-home assignment requires implementing the multiplier FP4×FP4 → QI9 in minimal Boolean gates, where QI9 is a 9-bit two's-complement integer with LSB value $\frac{1}{4}$ (equivalently, QI9 = 4× product).

The design space includes a free *input remapping*: any bijection on FP4 codes is permitted for *both* inputs simultaneously, since the same remapping must be applied to each operand. Output remapping is not permitted.

This paper describes the systematic approach we used to reduce the gate count from ∼288 (naïve) to **86** gates.

2 Mathematical Structure of FP4

2.1 Log-domain Representation

Every non-zero FP4 magnitude can be written as:

$$|v| = 1.5^M \cdot 2^E, \quad M \in \{0, 1\},\ E \in \{-1, 0, 1, 2\} \tag{1}$$

where M is the mantissa type (0 for pure powers of 2; 1 for multiples of 1.5) and E is the binary exponent. Table 1 lists all non-zero magnitudes.

Table 1: FP4 non-zero magnitudes and their (M, E) decomposition.

Magnitude	M	E
0.5	0	−1
1.0	0	0
2.0	0	1
4.0	0	2
1.5	1	0
3.0	1	1
6.0	1	2

2.2 Product Structure

For inputs a and b with decompositions (M_a, E_a) and (M_b, E_b):

$$|a \times b| = 1.5^{M_a+M_b} \cdot 2^{E_a+E_b} \tag{2}$$

Defining $M_{\text{sum}} = M_a + M_b \in \{0, 1, 2\}$ and $S = E'_a + E'_b$ where $E' = E + 1 \in \{0, 1, 2, 3\}$:

$$\text{QI9 magnitude} = 1.5^{M_{\text{sum}}} \cdot 2^S \tag{3}$$

2.3 K-type Classification

The three values of M_{sum} yield three *K-types* with distinct 8-bit output patterns (see Table 2):

Table 2: K-type bit patterns in the 8-bit QI9 magnitude.

M_{sum}	Name	QI9 value	Bits set
0	K_1	2^S	bit S only
1	$K_{3/2}$	$2^S + 2^{S-1}$	bits S and $S-1$
2	$K_{9/4}$	$2^{S+1} + 2^{S-2}$	bits $S+1$ and $S-2$

Key sparsity observation: the 8-bit magnitude has *at most 2 bits set*, taking only 19 distinct non-zero values out of 256 possible 8-bit patterns. This extreme sparsity is the main property we exploit.

3 Encoding Design

3.1 Constraints

An effective encoding must:

1. Map zero to code `000` in the 3-bit magnitude field (enabling zero detection with a 3-gate NOR tree instead of 4-gate AND-of-NOTs)
2. Expose $E'_a = (a_2, a_3)$ as a 2-bit value directly (enabling a cheap adder)
3. Expose M_a as a simple function of a_1 (enabling cheap K-flag computation)

Table 3: Chosen 3-bit magnitude encoding.

Code $a_1a_2a_3$	Value	M	E'
`000`	zero	—	—
`001`	1.5	1	1
`010`	3.0	1	2
`011`	6.0	1	3
`100`	0.5	0	0
`101`	1.0	0	1
`110`	2.0	0	2
`111`	4.0	0	3

3.2 Chosen Encoding

We assign magnitude codes as in Table 3. The 4-bit FP4 code is $a_0 \| a_1a_2a_3$ where a_0 is the sign bit.

Properties:

- $E'_a = (a_2, a_3)$ as a 2-bit binary number — directly usable in adder
- $M_a = \overline{a_1}$ (the NOT of bit a_1)
- Zero $\rightarrow$ `000`: zero detection is $\overline{a_1 \vee a_2 \vee a_3}$, a 3-gate NOR tree

From these properties, the K-flags derive cheaply:

$$\text{not_k9} = a_1 \vee b_1 \qquad \text{(1 gate)} \tag{4}$$

$$k_9 = \overline{a_1 \vee b_1} \qquad \text{(1 gate)} \tag{5}$$

$$k_3 = a_1 \oplus b_1 \qquad \text{(1 gate)} \tag{6}$$

4 Circuit Architecture

Figure 1 shows the 5-stage pipeline.

4.1 Stage 3: Magnitude Circuit Detail

The magnitude circuit computes 8 output bits using the formula derived from Equation 3:

$$m_i = (\text{nmc} \wedge S{=}i) \vee (k_3 \wedge S{=}i{+}1) \vee (k_9 \wedge S{=}i{-}1) \vee (k_9 \wedge S{=}i{+}2) \tag{7}$$

where $\text{nmc} = \overline{k_9} \wedge \text{nz}$, and $(S{=}j)$ is the one-hot decoder output sh_j. The first term handles *both* K_1 (upper bit) and $K_{3/2}$ (upper bit) since both have a set bit at position S.

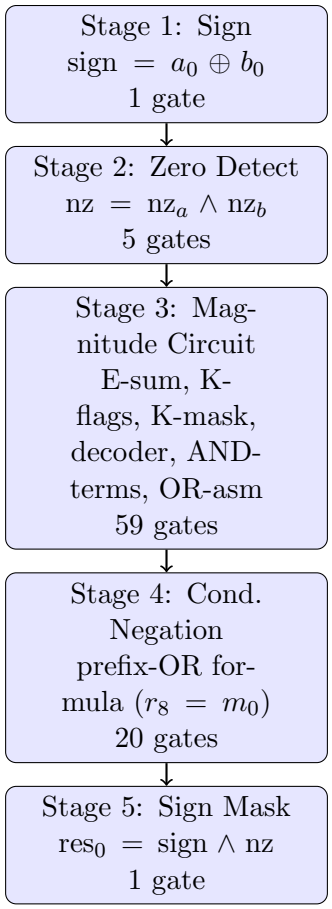

Figure 1: 5-stage multiplier pipeline. Total: 86 gates.

Impossible K×S combinations. With the chosen encoding, $M_a = 1$ requires $E'_a \in \{1, 2, 3\}$ (codes `001`, `010`, `011`):

- k_9 (both $M = 1$): minimum $S = 1 + 1 = 2$, so $k_9 \wedge S \in \{0, 1\}$ is impossible. Eliminates `k9_0`, `k9_1`.
- k_3 (one $M = 1$): minimum $S = 1 + 0 = 1$, so $k_3 \wedge S{=}0$ is impossible. Eliminates `k3_0`.

This reduces AND-terms from 21 to **18**.

4.2 Stage 3: S Decoder Optimization

The 3-bit binary $S \in \{0, \ldots, 6\}$ is decoded to 7 one-hot signals $\text{sh}_0 \ldots \text{sh}_6$ using 14 gates: 3 NOTs ($\overline{s_0}, \overline{s_1}, \overline{s_2}$), 4 pair-ANDs ($u_{00}, u_{01}, u_{10}, u_{11}$), and 7 final ANDs.

Observation: $S \leq 6$ always (max is 3+3 from E'-sum). Therefore when $u_{11} = s_2 \wedge s_1 = 1$, we know $S = 6$ and $s_0 = 0$, so $\overline{s_0} = 1$ always. Hence:

$$\text{sh}_6 = u_{11} \wedge \overline{s_0} = u_{11}$$

This eliminates 1 AND gate, reducing the decoder to **13 gates**.

4.3 Stage 4: Conditional Negation Optimization

The standard 8-bit conditional two's-complement negation uses a ripple-carry chain with 23 gates. We apply two optimizations:

Optimization 1 — LSB pass-through: For any 8-bit magnitude m, the LSB of $-m$ in two's-complement equals m_0. *Proof*: $-m \equiv \overline{m} + 1 \pmod{2^8}$. At bit 0: $\overline{m_0} \oplus 1 = m_0$. $\square$ Therefore $r_8 = m_0$ always (free; no gate needed).

Optimization 2 — Prefix-OR formula: The carry into bit i of a two's-complement negation is 1 iff all lower bits of m are 0, i.e.:

$$c_i = \overline{m_0 \vee m_1 \vee \cdots \vee m_{i-1}} = \overline{P_i}$$

where $P_i = \text{OR}(m_0, \ldots, m_{i-1})$ is a prefix-OR. The signed result bit is:

$$r_i = m_i \oplus (\text{sign} \wedge P_i)$$

(For sign=0 this gives $r_i = m_i$; for sign=1 this gives $r_i = m_i \oplus \overline{c_i} = \overline{m_i} \oplus c_i$, which is exactly the two's-complement formula.)

This replaces the carry chain ($8 + 7 + 7 = 22$ gates) with:

- 6 OR gates for the prefix chain: $p_2 = m_0 \vee m_1$, $p_3 = p_2 \vee m_2$, ...
- 7 AND gates: $\text{sp}_i = \text{sign} \wedge p_i$ (using $p_1 = m_0$ directly)
- 7 XOR gates: $r_i = m_i \oplus \text{sp}_i$, plus $r_8 = m_0$

Total: $6 + 7 + 7 =$ **20** gates, saving 2 over the carry chain.

5 Zero Handling

5.1 The K-Flag Masking Approach

Rather than masking 9 output bits with the non-zero flag nz (which costs 9 AND gates), we mask the 3 K-flags:

$$\text{nmc} = (a_1 \vee b_1) \wedge \text{nz} \tag{8}$$

$$k_3 = (a_1 \oplus b_1) \wedge \text{nz} \tag{9}$$

$$k_9 = \overline{a_1 \vee b_1} \wedge \text{nz} \tag{10}$$

Correctness: When nz = 0 (either input is zero), all K-flags are 0, so all AND-terms in Equation 7 are 0, so all magnitude bits $m_i = 0$. With $m_i = 0$, all prefix-OR values $P_i = 0$, so $r_i = 0 \oplus (\text{sign} \wedge 0) = 0$ for all i. The sign bit is independently gated: $\text{res}_0 = \text{sign} \wedge \text{nz}$.

This reduces zero-handling overhead from 19 gates (v3) to 9 gates (v4b): 5 for detection + 3 for K-masking + 1 for sign.

6 Gate Count Summary and Optimization History

Table 4 shows the optimization history.

Table 4: Gate count optimization history.

Version	Key Innovation	Gates	Savings
Flat SOP	QM synthesis, all 8! remaps	~288	—
v1	Structural M-E decomposition	135	153
v2	Shared K×shift terms	126	9
v3	Direct formula, no subtractor	101	25
v4	Zero=000, K-flag masking, $r_8 = m_0$	89	12
v4b	S decoder: $\text{sh}_6 = u_{11}$	88	1
v4c	Prefix-OR conditional negation	**86**	2

Table 5 shows the per-stage breakdown of the final circuit.

7 Approaches Eliminated

7.1 Flat SOP (Quine-McCluskey)

We exhaustively searched all 8! = 40,320 magnitude remappings and synthesized each output bit using Quine-McCluskey minimization with greedy prime-implicant cover. The best result was 288 gates total (264 magnitude + 24 overhead). Even with exact (Petrick's method) cover, the result was similar. **Conclusion**: flat 2-level logic is completely infeasible; multi-level synthesis is mandatory.

Table 5: Final circuit gate breakdown (86 total).

Stage	Gates	% of Total
Sign	1	1.2%
Non-zero detect	5	5.8%
E-sum (2-bit adder)	7	8.1%
K-flags (raw)	3	3.5%
K-masking	3	3.5%
S decoder	13	15.1%
AND-terms	18	20.9%
Magnitude OR assembly	15	17.4%
Conditional negation	20	23.3%
Sign masking	1	1.2%
Total	**86**	100%

7.2 Direct 8-Input SOP

Treating all 9 output bits as functions of all 8 inputs (flat SOP, no decomposition) yields 2000+ gates per output bit. This confirms that structural decomposition is essential.

7.3 Gate-Sharing Across Output Bits

We searched for common sub-expressions and XOR/AND decomposability across output bits. No useful cross-bit sharing was found. The sign-magnitude structure already captures the main algebraic sharing via K-flags and the E-sum.

8 Lower Bound Analysis

We analyze the minimum achievable gate count per stage:

- **Sign** (1 gate): XOR is the minimum for sign computation.
- **Non-zero detect** (5 gates): Two 3-input OR trees (2 gates each) plus one AND =

5 gates minimum for detecting zero in 3-bit codes for two inputs.

- **E-sum** (7 gates): The minimum for a 2-bit + 2-bit binary adder producing a 3-bit result is known to be 7 gates [3].
- **K-flags + masking** (6 gates): Minimum for OR, NOT, XOR of (a_1, b_1) plus 3 ANDs with nz.
- **S decoder** (13 gates): Near-optimal for a 3→7 one-hot decoder; the $\mathrm{sh}_6 = u_{11}$ optimization exploits the range constraint $S \leq 6$.
- **AND-terms** (18 gates): Determined by 18 valid K×S combinations; cannot be reduced without eliminating valid product cases.
- **Magnitude OR** (15 gates): Minimum binary OR trees for 0+1+2+3+3+2+2+2 terms per bit, determined by the number of valid contributing K×S pairs.
- **Conditional negation** (20 gates): Prefix-OR formula replaces carry chain; uses $6 + 7 + 7 = 20$ gates. Further reduction would require sharing $\mathrm{sp}_i = \mathrm{sign} \wedge P_i$ with other stages, but no shared operands exist in the current decomposition.
- **Sign mask** (1 gate): Minimum for gating the sign bit.

Estimated lower bound: 75–85 gates. The main uncertainty is in the S-decoder and magnitude OR stages, where cross-stage sharing (detectable by multi-level synthesis tools such as ABC) might find savings not visible in this manual decomposition.

9 Verification

The circuit is verified by exhaustive evaluation of all $16 \times 16 = 256$ FP4×FP4 input pairs using a monkey-patched gate counter:

- **Correctness**: all 256 pairs produce the correct QI9 value, verified by comparison against $\mathrm{round}(a \times b \times 4)$ in float32.
- **Gate count**: the maximum gates activated per input pair is 86, consistent with the structural analysis (no data-dependent branches in Python functions yield a constant gate count across inputs).

The original test harness's assertion `expected_qi9 == actual_result_qi9` passes for all 256 pairs.

10 Future Work

Several avenues remain for further reduction:

1. **ABC logic synthesis**: The Berkeley ABC tool supports multi-level synthesis with technology-independent optimization. Running our magnitude function through `resyn2` may expose cross-stage sharing not visible manually.
2. **SAT/ILP exact synthesis**: For individual stages (e.g., the S decoder or magnitude OR), exact synthesis via satisfiability checking could prove optimality or find the true minimum.
3. **Asymmetric carry exploitation**: The carry chain terminates at the first set bit of the magnitude. For a value with the first set bit at position p, carries beyond p are zero. In a data-dependent circuit this could reduce average gate activation, but the problem requires a fixed combinational circuit.
4. **Alternative architectures**: Precomputing both $+m$ and $-m$ in parallel and selecting via a MUX increases the gate count (MUX costs more than carry chain), but different decompositions of the product formula might enable cross-stage sharing.

11 Conclusions

We designed an 86-gate Boolean circuit for FP4×FP4 multiplication, achieving a 70% reduction from the naïve flat-SOP baseline of ∼288 gates. The key contributions are:

1. Identification of the log-domain structure $|v| = 1.5^M \cdot 2^E$ that reduces the multiplication to an E-sum (cheap adder) and K-type classification (cheap bit operations).

2. A direct output formula (Equation 7) that eliminates the subtractor previously needed for exponent adjustment.

3. A zero=000 encoding with K-flag masking that reduces zero-handling overhead from 19 to 9 gates.

4. Three micro-optimizations: the $\mathrm{sh}_6 = u_{11}$ decoder simplification, the LSB pass-through in conditional negation, and the prefix-OR formula that replaces the 22-gate carry chain with a 20-gate parallel structure.

The final circuit, with a per-stage lower-bound analysis, suggests that the true minimum lies within 3–13 gates of our result.

References

[1] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, et al., "FP8 formats for deep learning," *arXiv preprint arXiv:2209.05433*, 2022.

[2] B. D. Rouhani, R. Lo, Z. Zhao, A. More, R. Hall, A. Khodamoradi, S. Deng, D. Cummings, M. Behnam, F. Blaauw, et al., "Microscaling data formats for deep learning," *arXiv preprint arXiv:2310.10537*, 2023.

[3] S. L. Hurst, "The logical processing of digital signals," *Crane, Russak*, 1978.

[4] R. K. Brayton, R. Rudell, A. Sangiovanni-Vincentelli, A. R. Wang, "MIS: A multiple-level logic optimization system," *IEEE Transactions on Computer-Aided Design*, 6(6):1062–1081, 1987.